

CloudRamp: WebAssembly based FaaS for WebSockets

Peter Olsen Marcus Souders Amelia Lunning Russell Shaver

University of Minnesota - Twin Cities

Minneapolis, Minnesota

olse0321, soude025, lunni022, shave083@umn.edu

Abstract—Traditional Function-as-a-Service (FaaS) tools primarily work over HTTP and operate in containerized environments. While this model works for a wide range of cloud-based use cases, there are 2 significant limitations that often necessitate an alternative: large memory usage and high latency. In this paper, we propose a WebAssembly-based FaaS in order to better alleviate the restrictive needs of low memory usage latency in the context of websockets. We will demonstrate that our approach results in 10x increase in throughput and 15x decrease in memory usage compared to Docker containers, all while maintaining a simple user experience.

I. INTRODUCTION

Modern serverless cloud platforms have converged on a narrow set of design assumptions: HTTP as the dominant communication protocol, and containers as the standard execution environment. While this combination has proven workable at scale, it carries real costs. Container-based function execution introduces multiple factors increasing overhead, most notably memory overhead and communication latency. Meanwhile, restricting interfaces to HTTP leaves entire classes of applications, particularly those requiring persistent, bidirectional communication, as second-class citizens in the serverless model.

This paper challenges both assumptions. We build a serverless platform centered on WebSockets as the primary protocol, enabling low-latency, event-driven interactions that HTTP’s request-response model cannot efficiently support. Rather than containers, we execute user-defined functions inside WebAssembly (WASM) sandboxes — a technique already deployed in production by platforms such as Cloudflare Workers and Fly.io. WebAssembly offers near-native execution speed, strong isolation guarantees, and startup times orders of magnitude faster than container runtimes, making it well-suited for high-throughput function execution.

Together, these design choices open a different point in the serverless design space. We evaluate our system along two axes: *developer experience*, how naturally the platform supports building real applications, and *runtime performance* - latency and memory overhead compared to container-based baselines. Our results suggest that WebSocket-native, WASM-backed execution deserves serious consideration as a foundation for next-generation serverless infrastructure.

II. CURRENT APPROACHES

Current cloud providers do not provide FaaS systems with a WebSocket-forward approach. They typically rely primarily

on the HTTP protocol, and then allow users to string together multiple services if they wish to operate over different protocols. For example, AWS Lambdas can be directly invoked through a URL over the HTTP protocol, but supporting Lambdas with WebSockets requires a separate API Gateway service to glue the two together. In addition to these hassles, in order to create a true distributed real-time application, users need to string together other applications, such as a pub/sub provider, a key/value store, and a persistent database, among others.

Beyond protocol limitations, the internal runtime that executes user code presents many challenges. Historically, containers have been a very popular option for sandboxing untrusted user code. However, major cloud providers have moved towards creating their own proprietary code sandboxes. These include microVMs, such as AWS’s Firecracker [1], or user-space kernels that interrupt system calls, like Google’s gVisor [2]. Created to mitigate common runtime issues such as cold start, security, and portability, these systems power the majority of today’s FaaS services.

One important system to mention is Cloudflare’s workerd runtime, which powers their FaaS system Cloudflare Workers. It is notable because it allows for serverless functions to be uploaded as WebAssembly modules as opposed to raw source code. One library written by Cloudflare, the workers-rs library, allows for users to write code in Rust and compile it directly to WebAssembly to be run on their servers [3]. Because WebAssembly cannot access external system resources by default, most notably network resources for fetching, the workers-rs package defines a set of functions that the author can utilize to execute system calls.

III. USER EXPERIENCE

One main goal of our system is to make the user experience as easy as possible. Although Cloudflare’s workers-rs library compiles directly to WebAssembly to run natively on servers, not every developer is fluent in Rust. Therefore, we approach the problem of compiling directly to WebAssembly by utilizing a more developer-friendly technology—AssemblyScript [4]: a subset of TypeScript allowing for the low-level operations that WASM requires expressed with a language syntax that a majority of developers are comfortable with.

To utilize our package, the user will download our SDK from the standard npm package registry. Upon installation,

our package provides template code that allows the user to focus purely on writing code as opposed to system setup. Once ready, users can compile their code and upload their optimized WASM build directly to our website. Through the platform, the user can interact with their deployed instance in many ways, including: log viewing, code updating/deletion, an interactive WebSocket playground to send messages, and metric viewing. In future updates, we plan to include version control, allowing users to rollback their application to previous deployments if necessary.

IV. RUNTIME

CloudRamp uses the wazero [5] Go library to execute pre-compiled WASM modules. We selected wazero for its pure Go implementation with zero external dependencies, its support for WASI (a set of standardized system APIs for WASM), and its proven use in production environments. We discuss the benefits and tradeoffs of WASM-based sandboxing, along with implementation details of our custom runtime.

A. Runtime selection

We utilize a WebAssembly runtime to maintain simplicity while keeping latency and memory usage low. Executing code within a Docker environment would require setting up a communication socket between the server and container, loading code, and implementing a custom execution layer to intercept sensitive operations. Because WASM modules cannot access system resources by default, we provide users with an SDK that proxies networking and storage calls, communicating with our Go runtime in-memory for low-latency message passing. Our execution layer is inspired by operating systems trap instructions, where a user calls a sensitive operation, control is handed to our runtime to validate and execute the request, and the response is returned to the user.

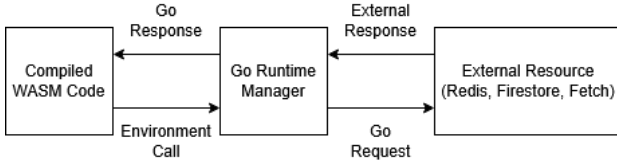


Fig. 1. Environment call execution

B. Malicious Code

One main issue with FaaS platforms is guarding against malicious user-uploaded code. By utilizing a WebAssembly runtime, user code executes in a sandboxed environment and can only access external resources through explicitly granted host functions. We expose a small set that are necessary for WebSocket applications, including external HTTP fetches, database operations, and key-value store access. Usage metrics are tracked per application, enabling enforcement of resource policies against apps that abuse external requests. This sandbox ensures that malicious code in one module cannot affect other modules or the host system. To prevent resource exhaustion, every execution is subject to a five-second

timeout and each module is limited to 640KB of memory (10 WASM pages), causing abusive code to fail gracefully with an error.

C. Communication Details

In order to communicate between the WebAssembly module and the coordinator, we defined a custom protocol, seen in figure 2 designed to minimize overhead while avoiding complex serialization. Our protocol is simply designed to pass an array of strings because we send only string data between the coordinator and WASM. While we could opt for a more expressive protocol, such as Redis' RESP protocol [6] that serializes more complex data types such as errors, strings, integers, and maps, it is too complex for the current needs of our project. Additionally, every message contains an indicator, either + or -, that signals if the following message is an error or not. Without this, we would not be able to securely pass error messages from the coordinator to the user code.

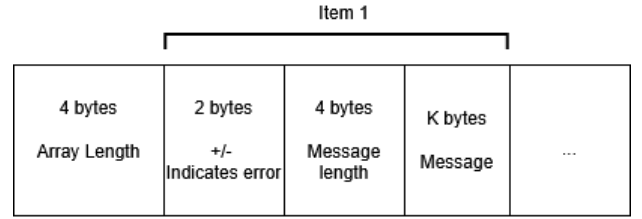


Fig. 2. Communication protocol diagram

To pass these messages between coordinator and WASM, their data is written directly to the memory of the module, and the receiving program is handed a pointer to the data. This avoids overhead present in many sandbox systems that would use a more complex system to communicate between processes, such as Unix domain sockets or pipes. One consequence of this approach is that formatting of strings must be handled explicitly. For example, AssemblyScript stores strings in UTF-16 encoding, as opposed to go's typical UTF-8 encoding, so conversion must be completed when communicating between the two. To ensure proper ordering of the converted bytes, all data transferred is written in little-endian.

D. Initial Approach: Single Module

Our initial approach executed requests from multiple clients on a single WASM module, using a lock to prevent concurrent memory access. This introduced two major issues: lock contention and memory corruption. Since only one invocation could run at a time, a hanging request would block the entire system until forcefully timed out. Even in the average case, a typical network request lasting multiple milliseconds would unnecessarily stall other goroutines. To address contention, we attempted releasing the lock during network requests. However, this introduced memory corruption errors: if a second goroutine modified the module's memory while the first was waiting on a network response, the first goroutine's memory state would be invalidated upon resuming.

E. Revised Approach: Module Pool

Our solution to these problems is a pool-based borrowing system. Upon loading a user's compiled module into memory, the system creates multiple copies of it, storing them in a pool. When a request arrives, a module will be borrowed from the pool. Access to the module will be isolated to the current goroutine for the duration of execution, and returned to the pool once execution is complete. This process can be observed in figure 3. One concern that we had with this approach is the memory usage. However, each compiled module typically uses 65 KB of memory, ranging up to 650 KB at max utilization, which only happens under very uncommon high system load, meaning that it is viable to maintain a pool of them because of their low memory footprint.

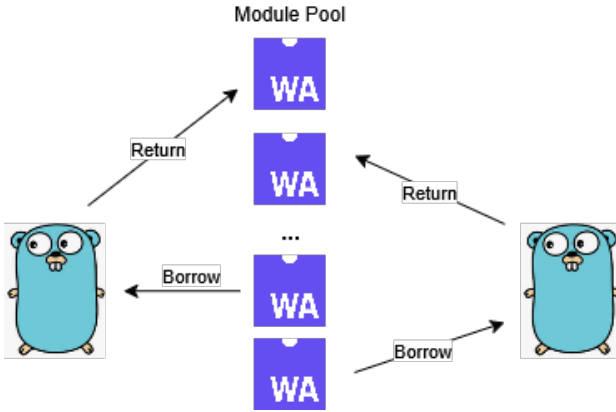


Fig. 3. Module pool design

V. SERVER DESIGN

The main purpose of our server is to act as an interface for users to call their uploaded functions. Because the purpose of our project is to provide FaaS over WebSocket connections, we created a WebSocket server that continuously listens for messages from connections and calls corresponding functions that the user has uploaded. These functions are designated to run on certain connection events, such as a user join, message, and leave.

A. Module storage

When a request arrives to a server replication for a module that is not yet loaded in memory, it is first fetched from an external cloud blob storage. This increases cold start latency, but is necessary as we cannot store every compiled module in memory within every server instance. Each server instance can store up to 50 modules in memory. Each of these modules is replicated with the pool structure as mentioned earlier. The server tracks the most recent request time of each module and evicts them in two ways: sweeping all idle modules, and evicting the least recently used. A module "sweep" is performed every 10 seconds, where the last request time of every module is examined, and if above 30 seconds since last request, it is evicted from the machine. This method of

eviction is more beneficial for our low-usage system, as the maximum number of modules in memory at one time we have recorded is three. If the system were to scale to more users, the LRU eviction policy would be more beneficial. Additionally, evicting modules to local file system first before removing them from the server entirely would improve future cold start times.

B. Communication Between Clients

One important aspect of this system is the communication between clients whose connections are located on different servers. Because these two clients are not on the same server and cannot communicate directly, we utilize a pub/sub based approach for inter-server communication. When a message is sent by client A connected to server A, for client B connected to server B, the message is published to Redis and received by all servers. However, our current naive approach results in many unnecessary message deliveries, since every message is sent to every server. For example, with N servers connected, a message for a single server will be delivered to $N-1$ servers that aren't the intended recipient. A possible solution to this issue would be to create a pub/sub channel for each server, reducing the number of unnecessary deliveries through only sending messages where necessary.

One small optimization to this problem that we include is checking if two client are co-located on the same machine before sending a message. If this is the case, the message is simply sent between the threads managing each connection, and delivered to the recipient. In practice, because our number of servers is minimal, this optimization greatly cuts down on external pub/sub usage.

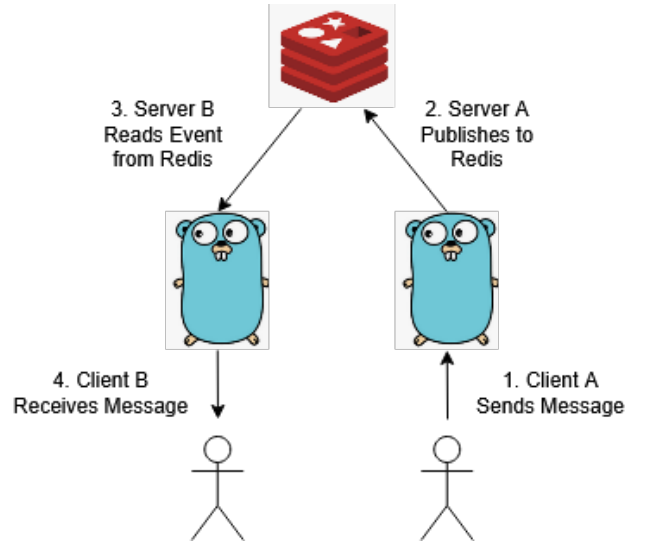


Fig. 4. Communication between two clients on different servers

C. Observability

We strive to maintain user visibility of their service through extensive logging and tracking of metrics. The metrics that we track are related to billing, such as egress bytes, requests to

external resources, and total connections to the server. Because we opted to store this information on Google Firestore, one issue was exceeding our write quotas. To circumvent this issue, we opted to batch service info on the local filesystem, and push it to Firestore asynchronously every minute. Additionally, because container services can preempt our system at any time, all batched information was pushed to Firestore upon the program receiving a shutdown signal, ensuring complete service visibility for our users.

D. Connection recovery

Another challenge addressed within our server is connection recovery. Our server assigns UUIDs to every connection it receives, and because WebSocket connections may be dropped at any time, our server ensures that no momentary loss of connection would completely eliminate a client's session. Therefore, upon client disconnect, a key/value pair of their IP and UUID is uploaded to Redis with a configurable five minute TTL. On every connection establishment, the server checks if the client's IP exists within Redis, and assigns their UUID to the corresponding value, if present. One problem with this method is that because NAT allows for multiple users to share the same IP within one network, it may assign the same UUID to multiple connections. This means that if a client within a protected network leaves and a new client from the same network joins, it may retain the first client's session information. Additionally, the IP/UUID pairs are not stored for a longer amount of time, because it would require a large amount of memory in our Redis store, which is limited to 30 MB, as we are operating on a free tier.

E. Security

In any online application, DOS attacks are inevitable and must be planned for. In order to prevent this, our server implements per-IP rate limiting. If a given IP sends more than 10,000 requests within a 30 second time window, they are sent a backoff message. The parameters for when to backoff a user were chosen because our server is capable of handling 40,000 requests a second. If a user has reached the rate limiting threshold, this means they have consumed at least 0.8% of the server's resources for 30 seconds. This information is stored locally on server replicas and pushed to Redis once every five seconds. This is done to avoid the overhead of rate-limiting every request through Redis while on a short enough interval to maintain consistency among servers. While it is possible that a user could send requests to multiple different servers within the five second time frame before state synchronization, our deployment on Google Cloud Run will automatically load balance requests to different servers, making this highly unlikely.

VI. EXPERIMENTS

All experiments for this project were run on a 2022 M2 MacBook Pro with 8gb of ram and 8 CPU cores. All test were written in go, utilizing the language's built in benchmarking and rate limiting tools. All tests that measured latency over

varying levels of throughput were run for 10 seconds at each level.

A. WASM vs. Docker

The main experiments we ran in this project were to compare the efficiency of a Docker-based user runtime versus a WebAssembly based runtime. In order to achieve this, we created a mini-replication of our WebAssembly system using Docker containers. The replication was stripped down to the core functionality of the system, which included code distribution, communication with the coordinator, code execution, and container pooling. The system was started with a pool of five containers maintaining communication channels with the coordinator through a TCP connection. All communication between the coordinator and container was done with a custom protocol over TCP to avoid as much communication overhead as possible. Upon a request to a user's code that has not been loaded, one of the open container was chosen with a round-robin selection policy and sent the code to parse and execute with the Node.JS runtime. Similarly to our system, if a request arrives for code that is not loaded, the least recently used module will be evicted. Once execution is complete, it sends an ACK notifying the coordinator. This ACK is not present in the WebAssembly approach, as the program knows execution is complete once the call to the user's method returns.

For the purposes of this test, we created a simple user code module for both the WASM and Docker implementations that calls an external method one time. By running benchmarks with this code, we test how well the systems respond to switches of control between the coordinator and the user code.

From the results of our tests, we can see that WebAssembly outperforms Docker in terms of throughput, as seen in figure 5 and memory usage, in figure 6. This is not unexpected, as the communication medium between our coordinator and Docker containers was TCP, which is much slower than writing directly to the WebAssembly module. Additionally, code execution in the Docker runtime was done with Node.JS, which generally executes slower than the Go wazero runtime.

However, this is mostly thanks to our desired workload. One main limitation of WebAssembly is that unless given explicit permission, it cannot access a multitude of system resources like the file system or network. In Docker containers, the user is given their own isolated view of the filesystem, allowing them to perform these more complex operations. One other major issue with our WebAssembly runtime is the package ecosystem. Because we compile user code with AssemblyScript, many of the common npm packages don't conform to its specifications and are therefore unusable. Because the Docker system runs code using Node.JS, there is much wider package support.

Although our WebAssembly runtime saw much higher throughput than the Docker implementation, it requires the tradeoff of explicitly creating a library of user functions to access permitted system resources, as well as greatly restricting the number of usable libraries. In contrast, the Docker runtime

performed slower but allowed for much more expressive user code.

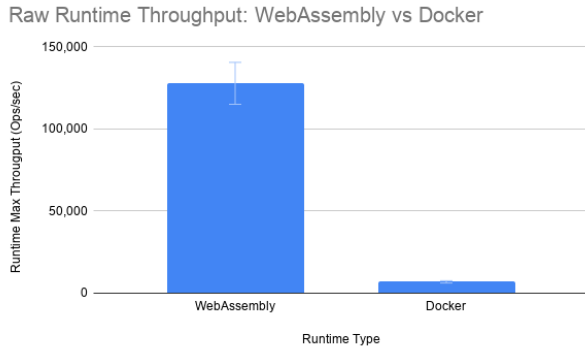


Fig. 5. Graph of isolated runtime performance

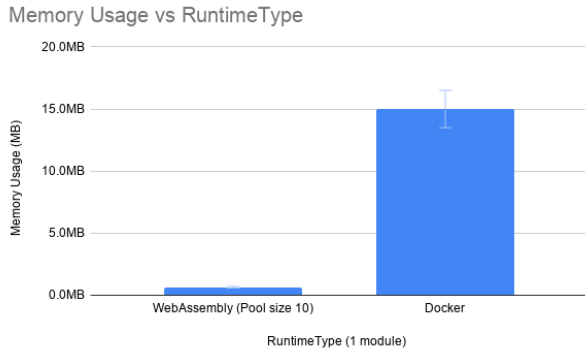


Fig. 6. Graph of memory usage for each runtime

B. System Latency Behavior

In order to understand the limits of our entire system, we benchmarked performance across two dimensions: single request latency over time, and request latency across varying throughput levels. Each test was run both locally and against our remote deployment on Google Cloud Run.

We benchmarked a simple user code module that responded to every inbound message, measuring round-trip latency. In typical production use, clients send messages to trigger server-side effects and receive asynchronous messages on events rather than synchronous replies. This test module therefore represents a more latency-sensitive workload than average, making our results a conservative estimate of real-world performance. Since production clients typically send messages without waiting for a synchronous response, the relevant latency is one-way, or approximately half of the round-trip times reported here.

Local testing of single client latency over time, as can be seen in figure 7, shows that the p50 latency hovered around 0.08 ms, with occasional spikes reaching 0.2ms, consistent with pauses for Go's garbage collector. Not pictured in the graph due to skewing of the y axis is the first request's

latency, which was 200ms, due to cold start module loading. We minimize cold-start through caching recently used modules on deployed server instances. One other option would be to deploy blob storage in the same region with Google Cloud Platform as opposed to our current AWS solution, reducing request latency for the module. In the online version of this test in figure 8, p50 latency hovered more around the 40ms range, with occasional spikes into the 100ms range, likely due to networking overhead.

Local testing of latency at varying levels of throughput found that the system reaches a ceiling of approximately 32,000 RPS. Below saturation, median latency remained around 0.08ms, climbing to 0.3ms at saturation, as can be seen in figure 9. Similarly, the online evaluation, noted in figure 10, found p50 latency in the 40ms range. The system did not reach saturation, as the network delay prevented a high enough throughput. Proper load testing of our deployed server would require a distributed set of clients generating enough load to overcome the network bottleneck.

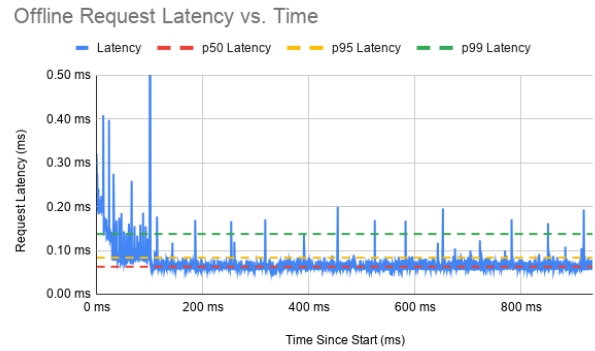


Fig. 7. Graph of latency versus time, tested locally

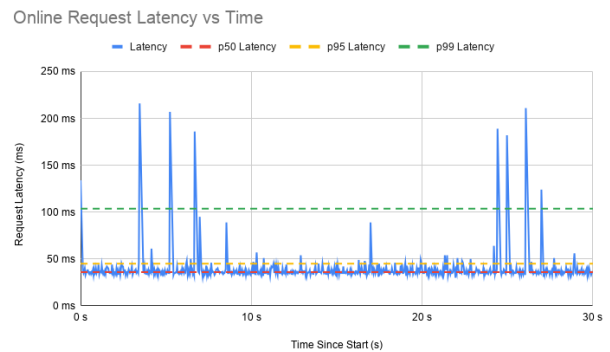


Fig. 8. Graph of latency versus time, tested on our deployment

Offline Latency vs Throughput (10 connections)

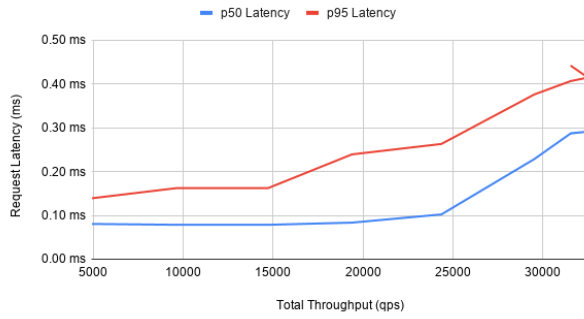


Fig. 9. Graph of latency versus throughput for 10 concurrent connections, tested locally

Online Latency vs Throughput (10 connections)

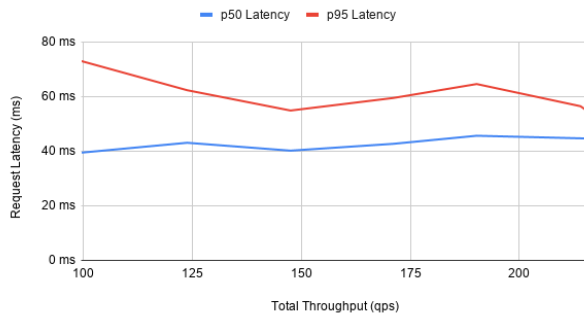


Fig. 10. Graph of latency versus throughput for 10 concurrent connections, tested on our deployment

REFERENCES

- [1] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, "Firecracker: Lightweight virtualization for serverless applications," in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. Santa Clara, CA: USENIX Association, Feb. 2020, pp. 419–434. [Online]. Available: <https://www.usenix.org/conference/nsdi20/presentation/agache>
- [2] E. G. Young, P. Zhu, T. Caraza-Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "The true cost of containing: A gVisor case study," in *11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 19)*. Renton, WA: USENIX Association, Jul. 2019. [Online]. Available: <https://www.usenix.org/conference/hotcloud19/presentation/young>
- [3] Cloudflare, "workers-rs: Write cloudflare workers in 100% rust via webassembly," <https://github.com/cloudflare/workers-rs>, 2026, accessed: 2026-05-04.
- [4] AssemblyScript Contributors, "Assemblyscript: A typescript-like language for webassembly," <https://www.assemblyscript.org/>, 2026, accessed: 2026-05-04.
- [5] Wazero Contributors, "Wazero: the zero dependency webassembly runtime for go developers," <https://wazero.io/>, 2026, accessed: 2026-05-04.
- [6] Redis Contributors, "Redis serialization protocol specification," <https://redis.io/docs/latest/develop/reference/protocol-spec/>, 2026, accessed: 2026-05-08.

VII. CONCLUSION

In summary, our WebAssembly based approach provides strong performance-based benefits in reducing used memory by 15x and increasing throughput by 15x compared to a traditional Docker-based application. In addition, these improvements complemented an ecosystem of developer-friendly tools, including a high-level language in AssemblyScript, a full-fledged web-based application for sandbox testing, and observability and security tooling. In the future, we plan to extend our implementation through quality of life improvements such as version control in deployments. Furthermore, expanding CloudRamp to a wider set of use cases, such as compute-heavy tasks, could be a promising direction for future work. Overall, our system demonstrates the value in treating WebSockets as a first-class protocol in the cloud computing space, and capitalizes on the performance and memory benefits of WebAssembly for sandboxed code execution.

ACKNOWLEDGMENTS

We would like to thank professor Ali Anwar for instructing the course on cloud computing and teaching us the system-design level intuition that makes a project like this possible.